

Lezione 1

Introduzione a R

Paolo Bosetti

Indice

1	Documenti Quarto (estensione .qmd)	2
1.1	Intestazione in YAML	2
1.2	Rendering: requisiti	3
1.3	Formattazione: Markdown	3
1.3.1	Questo è un titolo di livello 3	3
1.4	Elenchi puntati e numerati	4
1.5	Notebook: unire testo e codice	4
2	Richiamo basi di R	4
2.1	Vettori	4
2.2	Matrici	7
2.3	Tabelle (<code>data.frame</code>)	9
3	Funzioni di statistica descrittiva	11
4	Intermezzo: programmazione R	13
4.1	Funzioni pesonali	13
4.2	Tidyverse	15
4.2.1	Pipe	15
4.2.2	Tibble	17
5	Statistica inferenziale	18
5.1	Funzioni di distribuzione	18
5.2	Test di Student	19
6	Grafici con GGplot2	22
6.1	Preparazione dei dati	23
6.2	Scatterplot	24
6.3	Boxplot	30

6.4	Istogrammi	33
7	Funzioni di distribuzione e loro grafici	34
7.1	Densità di probabilità (PDF)	35
7.2	Ripartizione (CDF)	37
7.3	Quantile	39
8	Accesso a file	43
8.1	Caricamento dati	43
8.2	Salvataggio dati	43
9	Gestione tabelle	43
9.1	Filtraggio	43
9.2	Riordino	43
9.3	Selezione	43
9.4	Modifica	43
9.5	Raggruppamento	43
9.6	Trasposizione	43

1 Documenti Quarto (estensione .qmd)

Un documento Quarto è un file di testo che combina **testo formattato** (in Markdown) e **codice eseguibile** (in questo corso, codice R). Quando il documento viene *renderizzato*, il codice viene eseguito e i risultati (numeri, tabelle, grafici) vengono inseriti automaticamente nel documento finale (HTML, PDF, Word, ecc.).

1.1 Intestazione in YAML

Ogni documento inizia con un'intestazione delimitata da due linee ---, scritta in linguaggio YAML (un formato di testo del tipo **chiave: valore**). L'intestazione contiene i **metadati** del documento e le opzioni di rendering:

```
---
title: "Lezione 1"
subtitle: "Introduzione a R"
author: "Paolo Bosetti"
format: html
---
```

- `title`, `subtitle` e `author` definiscono titolo, sottotitolo e autore;
- `format` stabilisce il formato di output (`html`, `pdf`, `docx`, ...);

- altre opzioni utili sono ad esempio `toc: true` (indice dei contenuti), `date: today` (data di generazione) ed `embed-resources: true` (produce un unico file HTML autonomo).

Attenzione all'**indentazione**: in YAML è significativa, e le opzioni annidate vanno indentate con due spazi, ad esempio:

```
format:
  html:
    toc: true
```

Dopo i due punti lo spazio è **obbligatorio**!

1.2 Rendering: requisiti

Prima di renderizzare un documento Quarto, è necessario installare in R il *package* `rmarkdown`: dalla console di R eseguire il comando: `install.packages("rmarkdown")`.

Per creare documenti in HTML (più rapido, adatto ad una preview del contenuto), è poi sufficiente cliccare il pulsante “Preview” sulla toolbar del documento.

Per creare documenti in PDF (maggiore qualità), è necessario installare in R il package `tinytex`: dalla console di R eseguire il comando: `install.packages("tinytex")`, e successivamente eseguire `tinytex::install_tinytex()` per installare la distribuzione LaTeX completa. Questa operazione di setup va eseguita **una volta sola**. Dopodiché cliccare sul pulsante “Preview format...” e scegliere il formato “PDF (LaTeX)”

1.3 Formattazione: Markdown

Tipo di carattere: *corsivo*, **grassetto**, monospaziato (quest’ultimo per il codice).

1.3.1 Questo è un titolo di livello 3

I paragrafi vanno separati da una o più **linee vuote**.

Questo è il secondo *paragrafo*.

I commenti non compaiono nel documento finale. Per i titoli si usano uno o più **#** all’inizio della linea (uno per ogni livello).

1.4 Elenchi puntati e numerati

- Questo
- è un elenco
- puntato
 - con un
 - sottoelenco

Gli elenchi numerati si fanno con i numeri o le lettere:

1. Primo elemento
2. Secondo elemento: questo
è una ulteriore linea (a capo grazie al carattere \)
3. Terzo elemento
 - a. Sottoelemento
 - b. Altro sottoelemento
4. Quarto elemento

1.5 Notebook: unire testo e codice

Si può inserire un blocco di codice (*chunk*) così:

```
40 + 2
```

```
[1] 42
```

Il risultato dell'esecuzione compare subito sotto il blocco. Le scorciatoie di tastiera utili (in RStudio e Positron) sono:

- per inserire un blocco: Windows **Ctrl+Alt+I**, Mac **Cmd+Option+I**
- per eseguire la linea attiva (o la selezione): Windows **Ctrl+Invio**, Mac **Cmd+Invio**
- per eseguire l'intero blocco: Windows **Ctrl+Maiusc+Invio**, Mac **Cmd+Maiusc+Invio**

2 Richiamo basi di R

2.1 Vettori

Assegnazioni e vettori:

```
# assegnazione (si preferisce <- a =)
x <- 5
y = 3 # equivalente, ma meno usato
# vettori: la funzione c() concatena i valori
v <- c(10, 20, 30, 40, 50)
v2 <- 1:5 # sequenza di interi
v3 <- seq(0, 1, by = 0.25)
v
```

```
[1] 10 20 30 40 50
```

```
v2
```

```
[1] 1 2 3 4 5
```

```
v3
```

```
[1] 0.00 0.25 0.50 0.75 1.00
```

```
# indicizzazione di un vettore (in R gli indici partono da 1!)
v[2] # secondo elemento
```

```
[1] 20
```

```
v[c(1, 3)] # primo e terzo elemento
```

```
[1] 10 30
```

```
v[-1] # tutti tranne il primo
```

```
[1] 20 30 40 50
```

```
v[2:4] # dal secondo al quarto
```

```
[1] 20 30 40
```

```
# indicizzazione logica e filtraggio
v > 25      # vettore di valori logici
```

```
[1] FALSE FALSE  TRUE  TRUE  TRUE
```

```
v[v > 25]    # solo gli elementi maggiori di 25
```

```
[1] 30 40 50
```

```
v[v > 15 & v < 45] # combinazione di condizioni con & (AND) e | (OR)
```

```
[1] 20 30 40
```

Operazioni vettorializzate:

```
# operazioni vettore-scalare: lo scalare è applicato a ogni elemento
v * 2
```

```
[1] 20 40 60 80 100
```

```
v + 1
```

```
[1] 11 21 31 41 51
```

```
v^2
```

```
[1] 100 400 900 1600 2500
```

```
# operazioni vettore-vettore e riciclo degli elementi del vettore più corto
c(1, 2, 3, 4) + c(10, 20, 30, 40) # elemento per elemento
```

```
[1] 11 22 33 44
```

```
c(1, 2, 3, 4) + c(10, 20)      # il secondo vettore viene riciclato: 10, 20, 10, 20
```

```
[1] 11 22 13 24
```

Introduciamo una funzione utile nella forma più semplice (ne vedremo il dettaglio più avanti):
`sample(n)` ritorna i numeri da 1 a `n` in ordine casuale:

```
v1 <- sample(10)
v1
```

```
[1]  2  1 10  6  4  7  5  3  8  9
```

La lunghezza di un vettore si ottiene con la funzione `length()`. Per ottenere gli ultimi tre elementi di un vettore, ad esempio:

```
v <- sample(10)
v[length(v)-2:0]
```

```
[1] 3 9 4
```

Qui `2:0` è la sequenza 2, 1, 0, quindi gli indici sono `length(v)-2`, `length(v)-1` e `length(v)`. In alternativa, si può usare la funzione `tail(v, 3)`.

È anche possibile usare `length()` per **cambiare** la lunghezza di un vettore:

```
# allungare un vettore: i nuovi elementi sono NA (dato mancante)
length(v) <- 12
v
```

```
[1]  7 10  8  5  2  1  6  3  9  4 NA NA
```

```
# accorciare un vettore: gli elementi in eccesso vengono persi
length(v) <- 5
v
```

```
[1]  7 10  8  5  2
```

i Attenzione

Le *funzioni freccia* come `length() <-` sono una particolarità di R. Non tutte le funzioni hanno l'equivalente freccia: consultare l'help in linea di una funzione per scoprire se la versione freccia (cioè con assegnazione) esiste (es: `?length`).

2.2 Matrici

Le matrici si creano con la funzione `matrix()`:

```
# per righe:
m1 <- matrix(1:6, nrow = 2, byrow = TRUE)
m1
```

```
      [,1] [,2] [,3]
[1,]    1    2    3
[2,]    4    5    6
```

```
# per colonne (comportamento predefinito)
m2 <- matrix(1:6, nrow = 2)
m2
```

```
      [,1] [,2] [,3]
[1,]    1    3    5
[2,]    2    4    6
```

L'indicizzazione avviene come per i vettori, ma passando a [] due indici (riga e colonna). Un indice mancante significa “intera riga/colonna”:

```
m1[2, 3] # elemento in riga 2, colonna 3
```

```
[1] 6
```

```
m1[1, ] # intera prima riga
```

```
[1] 1 2 3
```

```
m1[, 2] # intera seconda colonna
```

```
[1] 2 5
```

```
m1[, c(1, 3)] # prima e terza colonna
```

```
      [,1] [,2]
[1,]    1    3
[2,]    4    6
```

Le operazioni aritmetiche tra matrici (+, -, *, /) avvengono elemento per elemento:

```
m1 * m2
```

```
      [,1] [,2] [,3]  
[1,]     1     6    15  
[2,]     8    20    36
```

```
m1 + 10
```

```
      [,1] [,2] [,3]  
[1,]    11    12    13  
[2,]    14    15    16
```

Per il prodotto matriciale si usa invece `%*%` (le dimensioni devono essere compatibili: colonne della prima = righe della seconda!). Per trasporre una matrice si usa `t()`:

```
m1 %*% t(m2) # (2x3) %*% (3x2) = matrice 2x2
```

```
      [,1] [,2]  
[1,]    22    28  
[2,]    49    64
```

2.3 Tabelle (data.frame)

Più utili delle matrici sono le **tabelle**, o **data.frame**: rappresentano matrici le cui colonne possono essere eterogenee (numeri, testo, valori logici, ...) e sono indicate da un **nome**.

```
# creazione di una tabella  
df <- data.frame(  
  A = 1:10,  
  B = sample(10),  
  Lettere = LETTERS[1:10]  
)  
df
```

```
      A  B Lettere  
1     1  4      A  
2     2  8      B  
3     3  1      C  
4     4  9      D
```

5	5	6	E
6	6	5	F
7	7	3	G
8	8	10	H
9	9	2	I
10	10	7	J

L'indicizzazione avviene esattamente come per le matrici con `[]`, oppure estraendo un'intera colonna **per nome** come vettore:

```
df[3, ]          # terza riga
```

```
  A B Lettere
3 3 1      C
```

```
df[, 2]          # seconda colonna
```

```
[1]  4  8  1  9  6  5  3 10  2  7
```

```
df[3, "B"]       # elemento singolo, con la colonna indicata per nome
```

```
[1] 1
```

```
df$B             # intera colonna B, come vettore
```

```
[1]  4  8  1  9  6  5  3 10  2  7
```

```
df[["B"]]        # equivalente a df$B
```

```
[1]  4  8  1  9  6  5  3 10  2  7
```

Nuove colonne possono essere create assegnandole per nome:

```
df$NuovaColonna <- df$A*2
head(df) # mostra solo le prime righe
```

	A	B	Lettere	NuovaColonna
1	1	4	A	2
2	2	8	B	4
3	3	1	C	6
4	4	9	D	8
5	5	6	E	10
6	6	5	F	12

La colonna può essere a sua volta indicizzata, e può essere usata per filtrare le righe della tabella (indicizzazione logica):

```
df[df$B>5,]
```

	A	B	Lettere	NuovaColonna
2	2	8	B	4
4	4	9	D	8
5	5	6	E	10
8	8	10	H	16
10	10	7	J	20

3 Funzioni di statistica descrittiva

Funzioni campionarie:

```
sum(df$B^2) # somma dei quadrati
```

```
[1] 385
```

```
mean(df$B) # media campionaria
```

```
[1] 5.5
```

```
sd(df$B) # deviazione standard campionaria
```

```
[1] 3.02765
```

```
var(df$B)      # varianza campionaria
```

```
[1] 9.166667
```

La somma dei quadrati degli scarti dalla media si calcola ad esempio con $\sum_{i=1}^N (y_i - \bar{y})^2$

```
ss <- sum((df$B - mean(df$B))^2)
ss
```

```
[1] 82.5
```

Si noti che la varianza campionaria è proprio $ss/(N-1)$:

```
ss / (length(df$B) - 1)
```

```
[1] 9.166667
```

```
var(df$B)
```

```
[1] 9.166667
```

Attenzione ai valori NA (dati mancanti): quasi tutte le funzioni statistiche restituiscono NA se anche un solo elemento è NA, a meno di indicare esplicitamente di ignorarli con `na.rm=TRUE`:

```
v5 <- c(3, 7, 4, NA, 9)
mean(v5)      # NA
```

```
[1] NA
```

```
mean(v5, na.rm=TRUE)      # ignora gli NA
```

```
[1] 5.75
```

```
mean(v5[!is.na(v5)])      # equivalente: filtra gli NA a mano
```

```
[1] 5.75
```

```
length(v5[!is.na(v5)])    # numero di dati validi
```

```
[1] 4
```

4 Intermezzo: programmazione R

4.1 Funzioni pesonali

Come si crea una funzione? le funzioni di R si creano con la parola chiave `function()`. Ad esempio, una funzione che moltiplica due numeri:

```
my_f <- function(a, b = 2) {  
  return(a * b)  
}
```

Se il blocco di una funzione ha una sola istruzione, si possono omettere le parentesi graffe. Inoltre, ogni funzione restituisce automaticamente l'ultimo valore calcolato, quindi si può omettere anche `return()`. Quindi le due definizioni seguenti sono equivalenti a quella sopra:

```
my_f <- function(a, b = 2) return(a * b)  
my_f <- function(a, b = 2) a * b
```

Nota

Come si vede, per essere usata una funzione deve essere **assegnata a una variabile** (in questo caso `my_f`). In R le funzioni sono **oggetti di prima classe**, quindi possono essere passate come argomenti ad altre funzioni, restituite come valori da altre funzioni, assegnate a variabili, ecc.

È ovviamente anche possibile ri-assegnare una funzione, come fatto nell'ultimo blocco.

Vediamo un esempio più complesso: una funzione che calcola la somma di un vettore. La funzione `length()` restituisce la lunghezza di un vettore, e il ciclo `for` permette di iterare su tutti gli elementi del vettore.

```
my_sum <- function(v) {  
  s <- 0  
  for (i in 1:length(v)) {  
    s <- s + v[i]  
  }  
}
```

```
    return(s)
  }

my_sum(1:3)
```

```
[1] 6
```

Il loop `for` può anche iterare direttamente sugli elementi del vettore, senza usare gli indici: vediamo come fare, aggiungendo anche il controllo sui valori `NA` (dati mancanti):

```
my_sum <- function(v, na.rm = FALSE) {
  s <- 0
  for (e in v) {
    if (!is.na(e)) s <- s + e
  }
  return(s)
}

my_sum(c(1:3, NA))
```

```
[1] 6
```

Oltre alla struttura condizionale `if|else`, è disponibile anche la funzione `ifelse()`

```
my_sum <- function(v, na.rm = FALSE) {
  s <- 0
  for (e in v) s <- s + ifelse(is.na(e), 0, e)
  return(s)
}

my_sum(c(1:3, NA))
```

```
[1] 6
```

Spesso succede di creare funzioni che possono essere ri-utilizzate in differenti documenti. In questo caso è utile salvare le funzioni in un file separato, detto *script* o *libreria*, e richiamarlo con la funzione `source()`. In questo modo le funzioni definite nel file esterno diventano disponibili nel documento corrente.

! Importante

L'estensione dei file script di R è `.R`, mentre l'estensione dei documenti Quarto è `.qmd`. In questo corso, il file `my_functions.R` contiene le funzioni `my_f()` e `my_sum()`, che sono richiamate nel documento `lezione1.qmd`.

Per caricare il nostro file “libreria”:

```
source("my_functions.R")
```

Welch's Test

nu: 19.9912363160818

t0: -2.00060634091529

p.value: 0.0592012600921777

Means: 9.92066301478032 10.5745271231138

💡 Consiglio

Di solito è opportuno caricare librerie e pacchetti all'inizio del documento, in un blocco di setup, come fatto all'inizio di questo documento. In questo modo le funzioni definite nel file esterno diventano disponibili in tutti i blocchi di codice del documento.

4.2 Tidyverse

Introduciamo l'uso di una libreria (*package*) molto utile e che sarà utilizzata estensivamente durante il corso: **Tidyverse**. Tidyverse è un insieme di pacchetti R che condividono una filosofia comune e sono progettati per lavorare bene insieme. La introdurremo gradualmente, ma per ora ci limitiamo a installarla e caricarla all'inizio del documento con `library(tidyverse)`. In questo modo tutte le funzioni dei pacchetti Tidyverse diventano disponibili nel documento.

! Importante

Per installare Tidyverse, nella console di R eseguire `install.packages("tidyverse")` (è necessario essere connessi a Internet). Una volta installata, la libreria va caricata all'inizio di ogni documento con `library(tidyverse)`.

4.2.1 Pipe

La prima funzionalità di Tidyverse che incontriamo è la *pipe* `%>%`, che permette di concatenare più funzioni in sequenza, rendendo più chiara l'esecuzione di più funzioni in sequenza.

Infatti, spesso succede di dover annidare funzioni, cioè usare il risultato di una funzione come argomento di un'altra funzione. Ad esempio potrei scrivere:

```
my_f(my_sum(c(1,3,2)), b=3)
```

```
[1] 18
```

L'espressione è corretta, ma non è molto chiara né leggibile: la prima funzione ad essere chiamata è infatti la più interna, e è necessario contare bene le parentesi per essere sicuri che i vari parametri siano passati alle funzioni giuste.

Per rendere più chiara l'espressione potrei usare variabili intermedie:

```
v <- c(1,3,2)
v_s <- my_sum(v)
v_r <- my_f(v_s, b=3)
```

Tuttavia, ciò “sporca” l'ambiente definendo molte variabili temporanee che non servono più dopo l'esecuzione e che pure occupano memoria (inutilmente!).

La *pipe* rende più chiara l'esecuzione di più funzioni in sequenza. Essa passa l'output della funzione a sinistra come primo argomento della funzione a destra. Quindi l'esempio precedente può essere scritto così:

```
c(1,3,2) %>%
  my_sum() %>%
  my_f(b=3)
```

```
[1] 18
```

Se voglio salvare il risultato, ovviamente:

```
res <- c(1,3,2) %>%
  my_sum() %>%
  my_f(b=3)
```

! Scorciatoia

Per inserire la pipe in RStudio, si può usare la scorciatoia **Ctrl + Shift + M** (o **Cmd + Shift + M** su macOS). Tuttavia attenzione: in R ci sono **due pipe**: la **pipe di**

Tidyverse %>% e la **pipe base |>**. La prima è più potente e flessibile, ma non è nativa di R (è definita nel pacchetto **magrittr**, che fa parte di Tidyverse). La seconda è nativa di R, ma meno potente. In questo corso useremo la pipe di Tidyverse. Per fare in modo che la scorciatoia da tastiera inserisca la pipe di Tidyverse, in RStudio aprire i setting di Positron con **Ctrl + ,** (o **Cmd + ,** su macOS), cercare “pipe” e selezionare %>% come scorciatoia.

4.2.2 Tibble

Un'altra struttura dati importante in Tidyverse è la **tibble**. Una **tibble** è una variante evoluta di un **data.frame**. Si crea una tibble con la funzione **tibble()**, che accetta come argomenti i nomi delle colonne e i valori corrispondenti, esattamente come **data.frame**.

In più, però, consente di definire una colonna come funzione di altre colonne, definite prima. Ad esempio:

```
tibble(  
  A = 1:10,  
  B = A * 2 # Questo NON FUNZIONEREBBE con data.frame()!  
)
```

```
# A tibble: 10 x 2
```

	A	B
	<int>	<dbl>
1	1	2
2	2	4
3	3	6
4	4	8
5	5	10
6	6	12
7	7	14
8	8	16
9	9	18
10	10	20

5 Statistica inferenziale

5.1 Funzioni di distribuzione

In generale R dispone di numerose funzioni di distribuzione, i cui nomi seguono questo pattern:
`[r|p|q|d] [nome] ()`

Ad esempio, per la distribuzione normale, le funzioni sono

- `rnorm()`: genera numeri casuali normali
- `pnorm()`: probabilità di un dato valore (CDF)
- `qnorm()`: quantile di una data probabilità
- `dnorm()`: PDF di un dato valore

Le distribuzioni più comuni sono:

- `norm`: normale
- `unif`: uniforme
- `t`: T di Student
- `chisq`: Chi quadro
- `f`: F di Snedecor
- `binom`: binomiale (discreta)
- `pois`: Poisson (discreta)
- `geom`: geometrica (discreta)

Ad esempio, per generare 10 valori casuali da una distribuzione normale con media 3 e deviazione standard 5:

```
set.seed(1)
rnorm(10, mean=3, sd=5)
```

```
[1] -0.1322691  3.9182166 -1.1781431 10.9764040  4.6475389 -1.1023419
[7]  5.4371453  6.6916235  5.8789068  1.4730581
```

Si noti che `set.seed()` serve a fissare il seme del generatore di numeri casuali, in modo che i risultati siano riproducibili. Se si esegue più volte lo stesso codice senza fissare il seme, i valori generati saranno diversi ogni volta:

```
rnorm(10, mean=3, sd=5)
```

```
[1] 10.5589058  4.9492162 -0.1062029 -8.0734994  8.6246546  2.7753320
[7]  2.9190487  7.7191811  7.1061060  5.9695066
```

Per le funzioni PDF, CDF e quantile, ritardiamo l'illustrazione a quando avremo imparato a realizzare grafici in R, in modo da poter visualizzare le distribuzioni.

5.2 Test di Student

Vediamo come realizzare un test di Student per confrontare due campioni. Creiamo due campioni casuali e applichiamo la funzione `t.test()`:

```
set.seed(1)
c1 <- rnorm(12, 10, 1.1)
c2 <- rnorm(10, 15.2, 1.1)

t.test(c1, c2)
```

Welch Two Sample t-test

```
data:  c1 and c2
t = -11.744, df = 17.068, p-value = 1.336e-09
alternative hypothesis: true difference in means is not equal to 0
95 percent confidence interval:
 -6.082240 -4.230104
sample estimates:
mean of x mean of y
 10.29550  15.45167
```

! Attenzione!

Di default, `t.test()` assume che i due campioni siano eteroschedastici, e effettua quindi un test di Welch! consultare l'help in linea di `t.test()` per maggiori dettagli. Per forzare l'ipotesi di varianze uguali, usare l'opzione `var.equal=TRUE`.

Per un test a un campione è opportuno specificare il valore di μ_0 nell'ipotesi nulla, con l'opzione `mu`:

```
t.test(c1, mu=9)
```

One Sample t-test

```
data:  c1
t = 5.0322, df = 11, p-value = 0.0003826
```

```
alternative hypothesis: true mean is not equal to 9
95 percent confidence interval:
  9.728879 10.862124
sample estimates:
mean of x
 10.2955
```

Per verificare l'omoschedasticità dei due campioni, si può utilizzare il test della varianza:

```
var.test(c1,c2)
```

F test to compare two variances

```
data:  c1 and c2
F = 0.62869, num df = 11, denom df = 9, p-value = 0.4618
alternative hypothesis: true ratio of variances is not equal to 1
95 percent confidence interval:
 0.160704 2.255662
sample estimates:
ratio of variances
 0.6286861
```

Il p -value grande consiglia di accettare l'ipotesi nulla di varianze uguali, quindi possiamo forzare l'ipotesi di varianze uguali nel test di Student:

```
t.test(c1, c2, var.equal=T)
```

Two Sample t-test

```
data:  c1 and c2
t = -12.002, df = 20, p-value = 1.355e-10
alternative hypothesis: true difference in means is not equal to 0
95 percent confidence interval:
 -6.052308 -4.260036
sample estimates:
mean of x mean of y
 10.29550  15.45167
```

💡 Esercizio

Provare a cambiare gli argomenti `var.equal`, `mu`, e `conf.level` di `t.test()` osservando come cambia l'output e cercando di commentare le differenze.

Le funzioni di test oltre che stampare il report ritornano anche un oggetto risultato che può essere salvato per accedere programmaticamente ai valori di interesse (statistica del test, p-value, intervallo di confidenza, ecc.). Ad esempio:

```
vt <- var.test(c1,c2)
attributes(vt) # mostra il contenuto di vt
```

```
$names
[1] "statistic"  "parameter"  "p.value"     "conf.int"    "estimate"
[6] "null.value" "alternative" "method"      "data.name"

$class
[1] "htest"
```

In questo modo possiamo accedere al p-value del test della varianza con `vt$p.value`. Possiamo quindi usare questo valore per decidere se forzare o meno l'ipotesi di varianze uguali nel test di Student:

```
t.test(c1, c2, var.equal=(vt$p.value > 0.05))
```

Two Sample t-test

```
data:  c1 and c2
t = -12.002, df = 20, p-value = 1.355e-10
alternative hypothesis: true difference in means is not equal to 0
95 percent confidence interval:
 -6.052308 -4.260036
sample estimates:
mean of x mean of y
 10.29550  15.45167
```

i Oggetti composti e liste

Le funzioni di test ritornano un oggetto che è in realtà una **lista**. Una lista è una collezione eterogenea di valori e oggetti identificati per nome.

Una lista può essere creata con la funzione `list()`:

```
l <- list(n1=10, n2=1:4, n3="test", n4=vt)
```

Per accedere ai contenuti della lista si usa lo stesso operatore `$` che abbiamo visto per le tabelle: `l$n1` restituisce il numero 10, `l$n2` restituisce il vettore `1:4`, `l$n3` restituisce la stringa `"test"`, e `l$n4` restituisce l'oggetto risultato del test della varianza. Per accedere al p-value del test della varianza, quindi, possiamo scrivere:

```
l$n4$p.value
```

```
[1] 0.4617866
```

Agli elementi di una lista si può accedere anche per posizione:

```
l[[2]]
```

```
[1] 1 2 3 4
```

6 Grafici con GGplot2

Il pacchetto `ggplot2` (anch'esso parte di Tidyverse, quindi già caricato con `library(tidyverse)`) è lo strumento più diffuso in R per realizzare grafici. Si basa su un'idea precisa, la cosiddetta **grammatica dei grafici** (*grammar of graphics*): ogni grafico, per quanto complesso, può essere descritto combinando pochi elementi indipendenti:

- i **dati**, cioè una tabella (`tibble` o `data.frame`);
- le **mappature estetiche** (*aesthetics*), che stabiliscono quale colonna della tabella controlla quale proprietà visiva del grafico (posizione in x e y , colore, dimensione, forma, ...);
- le **geometrie**, cioè il tipo di oggetto grafico con cui i dati vengono rappresentati (punti, linee, barre, box, ...);
- altri elementi opzionali: etichette, scale, temi grafici, suddivisione in sotto-grafici, ecc.

Un grafico si costruisce quindi **per strati** (*layer*), aggiungendo un elemento alla volta con l'operatore `+`. Questo approccio è molto diverso da quello delle funzioni grafiche di base di R (come `plot()`), ma una volta compreso permette di realizzare grafici anche molto elaborati con poche righe di codice.

! %>% oppure +?

La pipe %>% serve a passare i dati *dentro* la funzione `ggplot()`; da quel momento in poi gli strati del grafico si sommano con `+`. Un errore molto comune è usare %>% al posto di `+` (o viceversa) tra gli strati del grafico: R in questo caso restituisce un errore.

6.1 Preparazione dei dati

`ggplot2` lavora sempre con tabelle in cui **ogni colonna è una variabile** e **ogni riga è un'osservazione** (questo formato si chiama *tidy data*, da cui il nome Tidyverse). Prepariamo quindi una tabella di esempio, che simula una grandezza *y* misurata nel tempo e affetta da un disturbo casuale:

```
set.seed(0)
data <- tibble(
  time = seq(0, 20, length.out=30),
  yn    = 2 + 0.7*time + 0.05*time^2,      # valore nominale
  y     = yn + rnorm(length(time), 0, 2.5), # misura con disturbo
  c     = sample(length(time)),            # numero casuale
  when  = "sera"                          # colonna costante
)
data
```

```
# A tibble: 30 x 5
   time    yn     y     c when
  <dbl> <dbl> <dbl> <int> <chr>
1 0      2     5.16     7 sera
2 0.690  2.51  1.69    19 sera
3 1.38   3.06  6.39    10 sera
4 2.07   3.66  6.84     6 sera
5 2.76   4.31  5.35    24 sera
6 3.45   5.01  1.16    14 sera
7 4.14   5.75  3.43     2 sera
8 4.83   6.54  5.81    13 sera
9 5.52   7.38  7.37    18 sera
10 6.21   8.27 14.3    25 sera
# i 20 more rows
```

Alcune osservazioni sul codice:

- `seq(0, 20, length.out=30)` genera 30 valori equispaziati tra 0 e 20;

- all'interno di `tibble()` è possibile usare le colonne **già definite** per calcolare le successive: `yn` dipende da `time`, e `y` dipende da `yn`. Questo non è possibile con `data.frame()`;
- `sample(n)` restituisce una permutazione casuale degli interi da 1 a `n`;
- un valore singolo (come `"sera"`) viene **ripetuto** automaticamente su tutte le righe.

Possiamo poi modificare una parte di una colonna con la consueta indicizzazione dei vettori:

```
data$when[1:10] <- "mattina" # modifico solo i primi 10 elementi
data
```

```
# A tibble: 30 x 5
   time    yn    y    c when
  <dbl> <dbl> <dbl> <int> <chr>
1 0      2    5.16     7 mattina
2 0.690  2.51  1.69    19 mattina
3 1.38   3.06  6.39    10 mattina
4 2.07   3.66  6.84     6 mattina
5 2.76   4.31  5.35    24 mattina
6 3.45   5.01  1.16    14 mattina
7 4.14   5.75  3.43     2 mattina
8 4.83   6.54  5.81    13 mattina
9 5.52   7.38  7.37    18 mattina
10 6.21   8.27 14.3    25 mattina
# i 20 more rows
```

Ora la colonna `when` è una variabile **categorica** (o *discreta*) con due livelli, `"mattina"` e `"sera"`, mentre `time`, `yn` e `y` sono variabili **continue**. Questa distinzione è importante, perché `ggplot2` sceglie automaticamente il modo di rappresentare una variabile (ad es. la scala di colori) in base al suo tipo.

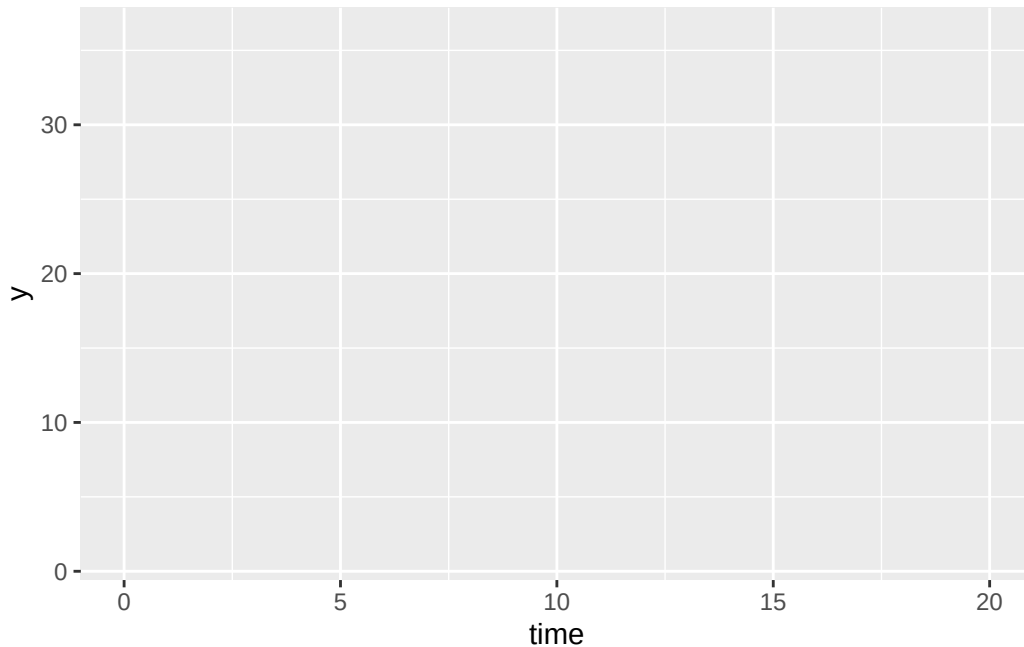
6.2 Scatterplot

Le tre funzioni fondamentali di `ggplot2` sono:

- `ggplot()` per creare l'ambiente e caricare i dati da una tabella
- `aes()` per associare le colonne della tabella agli elementi *estetici* del grafico (es. assi, colori, tipi di linea, ecc.)
- `geom_*()` per creare uno o più *layer*, ciascuno con la sua componente grafica (il plot vero e proprio)

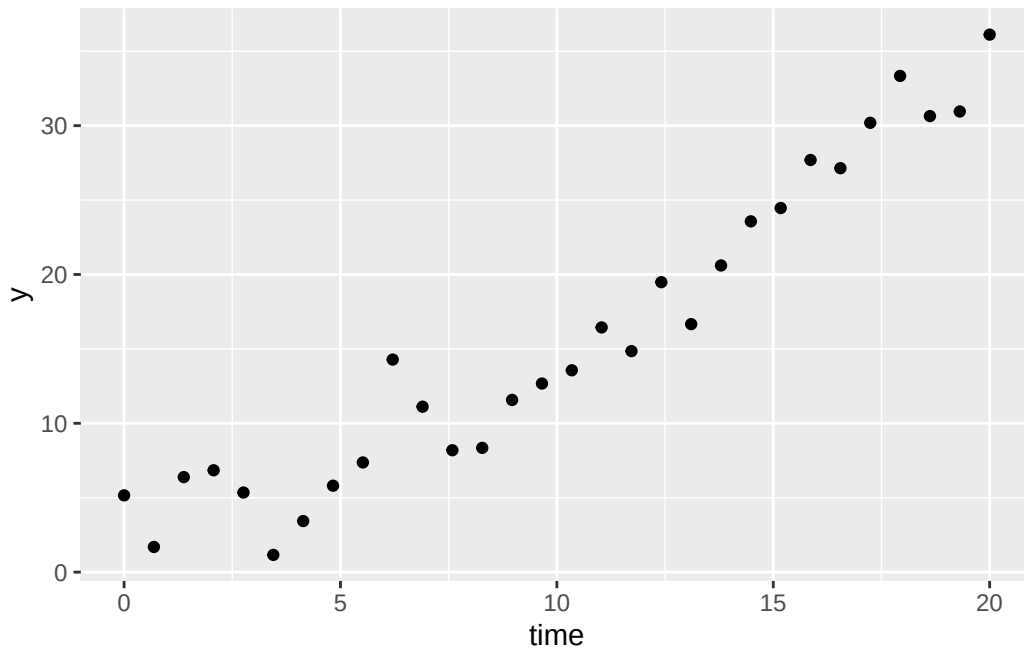
Vediamo come il grafico si costruisce passo passo. La sola chiamata a `ggplot()` con le estetiche crea un grafico **vuoto**: gli assi sono già dimensionati sui dati, ma non c'è ancora nessuna geometria che li rappresenti:

```
data %>%  
  ggplot(aes(x=time, y=y))
```



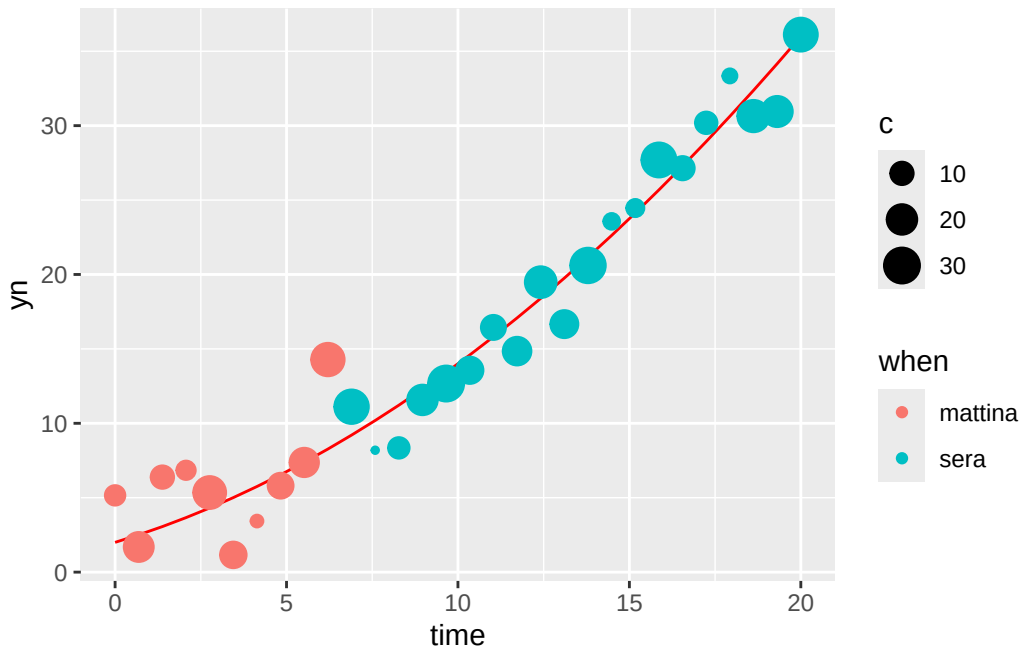
Aggiungendo uno strato con `geom_point()` otteniamo un diagramma a dispersione (*scatter-plot*):

```
data %>%  
  ggplot(aes(x=time, y=y)) +  
  geom_point()
```



Possiamo aggiungere quanti strati vogliamo. Ad esempio, una linea per il valore nominale e dei punti per le misure, colorati in base alla colonna **when** e con dimensione proporzionale alla colonna **c**:

```
data %>%  
  ggplot(aes(x=time)) +  
  geom_line(aes(y=yn), color="red") +      # grafico a linea  
  geom_point(aes(y=y, color=when, size=c)) # grafico a dispersione
```



Si noti che le estetiche possono essere definite in due posizioni:

- dentro `ggplot()`: valgono per **tutti** gli strati del grafico (qui `x=time`, comune a linea e punti);
- dentro una singola `geom_*()`: valgono **solo per quello strato** (qui `y=yn` per la linea e `y=y` per i punti).

Le legende vengono create automaticamente per ogni estetica associata a una colonna (qui `color` e `size`).

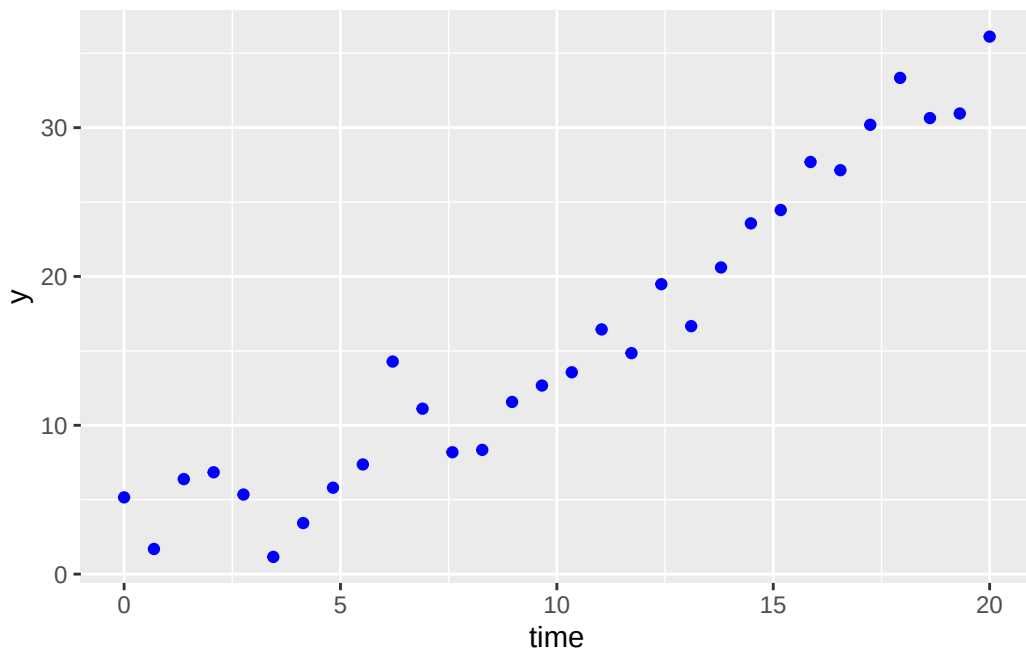
! Importante

Le **estetiche** disponibili per ogni geometria sono elencate nell'help.

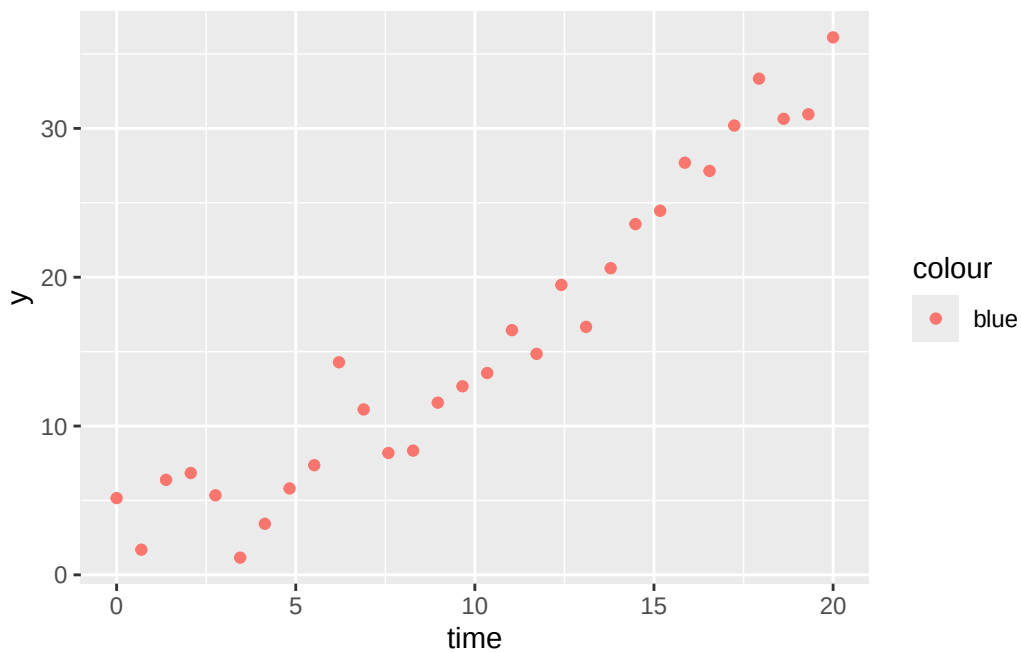
Un'estetica può essere passata in `aes()`, e allora richiede una variabile e viene fatta variare di conseguenza. Oppure può essere passata come argomento della geometria, e allora deve essere una costante e vale per l'intera geometria.

La differenza è importante e fonte di errori frequenti. Confrontiamo:

```
data %>%
  ggplot(aes(x=time, y=y)) +
  geom_point(color="blue")      # costante: tutti i punti blu
```



```
data %>%
  ggplot(aes(x=time, y=y)) +
  geom_point(aes(color="blue")) # SBAGLIATO: "blue" è trattato come un dato!
```

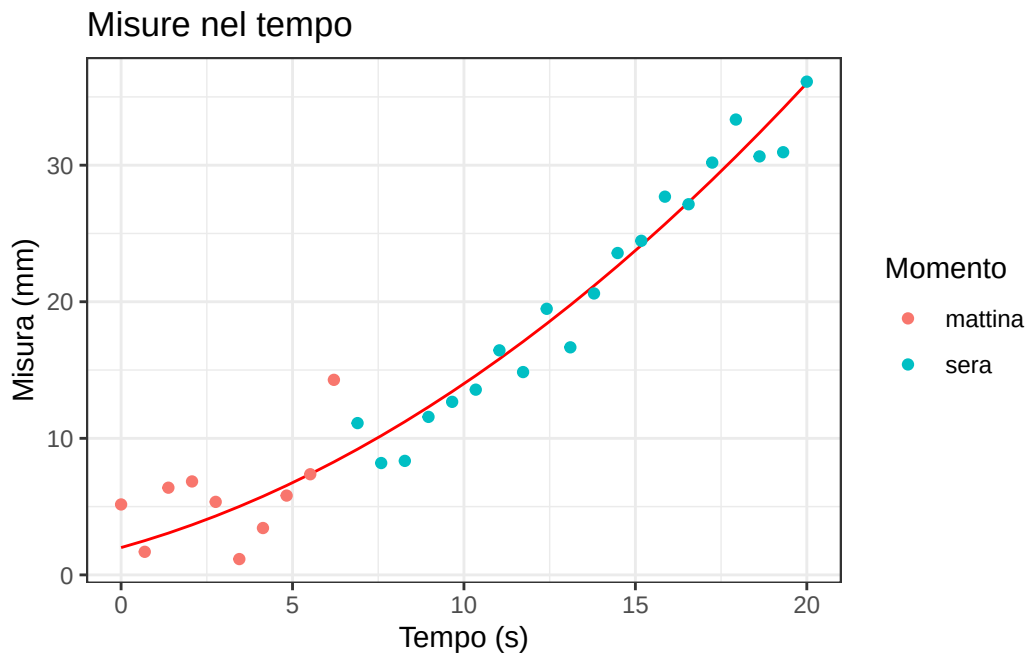


Nel secondo caso `ggplot2` interpreta "blue" come una variabile categorica con un solo livello (di nome "blue"), le assegna il primo colore della tavolozza predefinita (rosso) e crea una legenda inutile.

Infine, un grafico è un oggetto R come gli altri: può essere salvato in una variabile e arricchito in un secondo momento aggiungendo altri strati, ad esempio etichette (`labs()`) e un tema grafico (`theme_*()`):

```
p <- data %>%
  ggplot(aes(x=time)) +
  geom_line(aes(y=yn), color="red") +
  geom_point(aes(y=y, color=when))

p +
  labs(
    title = "Misure nel tempo",
    x = "Tempo (s)",
    y = "Misura (mm)",
    color = "Momento" # titolo della legenda dei colori
  ) +
  theme_bw() # cambia il "tema", cioè lo stile grafico
```



6.3 Boxplot

Il **boxplot** è un grafico che riassume la distribuzione di un campione: la linea centrale della scatola è la **mediana**, i lati della scatola sono il primo e il terzo **quartile** (quindi la scatola contiene il 50% centrale delle osservazioni), i “baffi” si estendono fino all’osservazione più estrema che dista non più di 1.5 volte l’intervallo interquartile dalla scatola, e le eventuali osservazioni oltre i baffi sono rappresentate come punti isolati (possibili *outlier*). È lo strumento ideale per confrontare visivamente più campioni.

Creiamo una tabella con tre campioni:

```
set.seed(123)
data <- tibble(
  c1 = rnorm(10, mean=12, sd=3),
  c2 = rnorm(10, mean=14, sd=2.2),
  c3 = rnorm(10, mean=9, sd=3.5)
)
data
```

```
# A tibble: 10 x 3
      c1      c2      c3
  <dbl> <dbl> <dbl>
1  10.3  16.7   5.26
2  11.3  14.8   8.24
3  16.7  14.9   5.41
4  12.2  14.2   6.45
5  12.4  12.8   6.81
6  17.1  17.9   3.10
7  13.4  15.1  11.9
8   8.20  9.67   9.54
9   9.94 15.5   5.02
10 10.7  13.0  13.4
```

Questa tabella è in formato **largo** (*wide*): ogni campione occupa una colonna diversa. Il boxplot però vuole in ascissa il nome del campione (variabile discreta) e in ordinata il valore delle osservazioni (variabile continua): ci serve cioè una tabella in formato **lungo** (*long*), con due colonne, una con i nomi dei campioni e una coi rispettivi valori. Questa operazione può essere fatta con `pivot_longer()`, che ha tre argomenti principali:

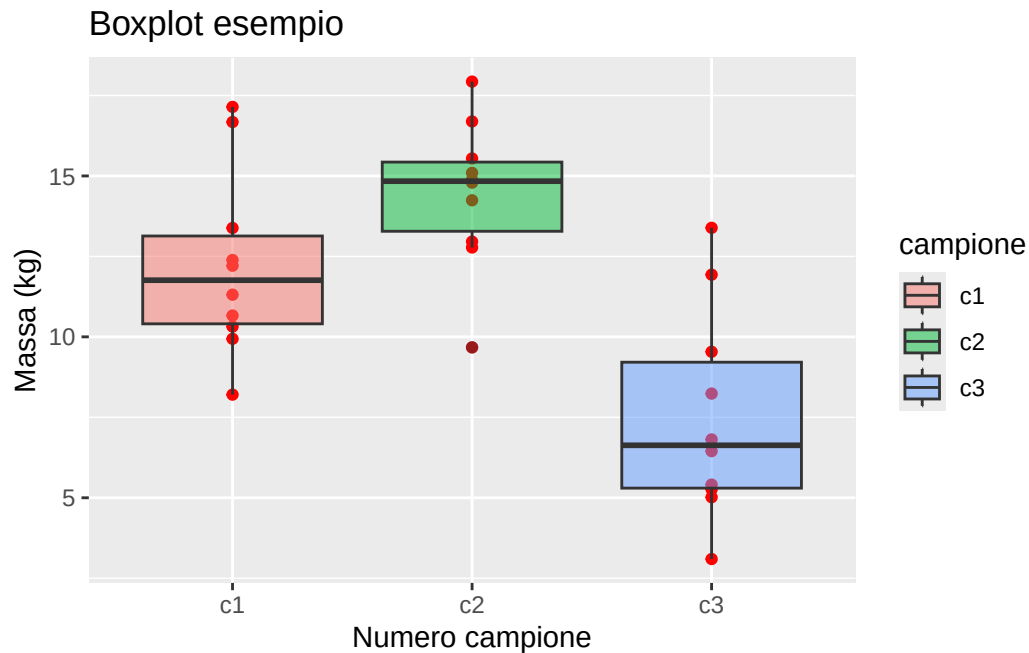
- le colonne da “impilare” (qui `c1:c3`, cioè tutte le colonne da `c1` a `c3`);
- `names_to`: il nome della nuova colonna che conterrà i **nomi** delle vecchie colonne;
- `values_to`: il nome della nuova colonna che conterrà i **valori**.

```
data %>%
  pivot_longer(c1:c3, names_to="campione", values_to = "valore")
```

```
# A tibble: 30 x 2
  campione valore
  <chr>      <dbl>
1 c1        10.3
2 c2        16.7
3 c3         5.26
4 c1        11.3
5 c2        14.8
6 c3         8.24
7 c1        16.7
8 c2        14.9
9 c3         5.41
10 c1        12.2
# i 20 more rows
```

La tabella risultante ha 30 righe (3×10) e 2 colonne. Ora possiamo passarla direttamente a `ggplot()`:

```
data %>%
  pivot_longer(c1:c3, names_to="campione", values_to = "valore") %>%
  ggplot(aes(x=campione)) +
  geom_point(aes(y=valore), color="red") +
  geom_boxplot(aes(y=valore, fill=campione), alpha=0.5) +
  labs(title="Boxplot esempio", x="Numero campione", y="Massa (kg)")
```



Si notino:

- la sovrapposizione di due strati: i punti (le singole osservazioni) e i boxplot;
- l'estetica `fill`, che colora il **riempimento** della scatola (mentre `color` colorava il **bordo**);
- l'argomento `alpha`, che regola la trasparenza (da 0, trasparente, a 1, opaco), permettendo di vedere i punti sotto le scatole.

i Specificare le colonne in `pivot_longer()`

Le colonne da impilare possono essere indicate in vari modi: con un intervallo (`c1:c3`), con un elenco (`c(c1, c2, c3)`), per esclusione (`-x` significa "tutte le colonne tranne x"), oppure con funzioni di selezione come `starts_with("c")`. Lo stesso vale per molte altre funzioni di Tidyverse.

💡 Esercizio

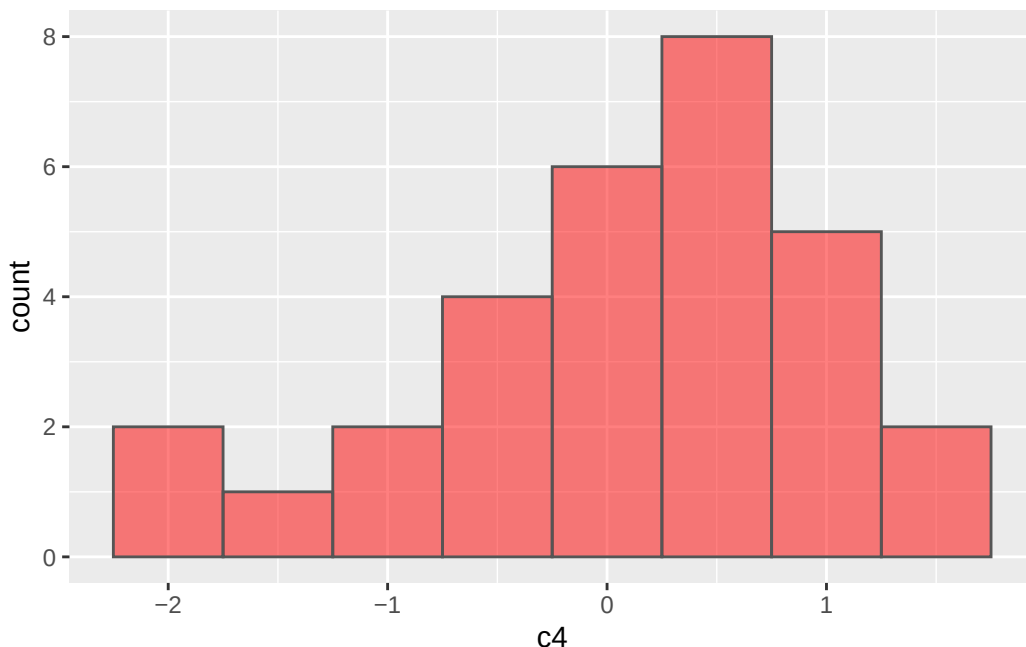
Consultando l'help per `?tidyr::pivot_wider()` provare a tornare alla tabella originale (10 righe e tre colonne), invertendo l'operazione di `pivot_longer()`.

6.4 Istogrammi

L'**istogramma** rappresenta la distribuzione di frequenza di un singolo campione: l'asse x viene suddiviso in intervalli (*bin*) e l'altezza di ciascuna barra è il numero di osservazioni che cadono in quell'intervallo. Un istogramma non richiede una tabella, perché l'origine dei dati è un solo vettore: in questo caso chiamiamo `ggplot()` senza argomenti e passiamo il vettore direttamente all'estetica:

```
set.seed(1)
c4 <- rnorm(30)

ggplot() +
  geom_histogram(aes(x=c4),
    binwidth=0.5,
    color=gray(1/3),
    fill="red", alpha=0.5)
```

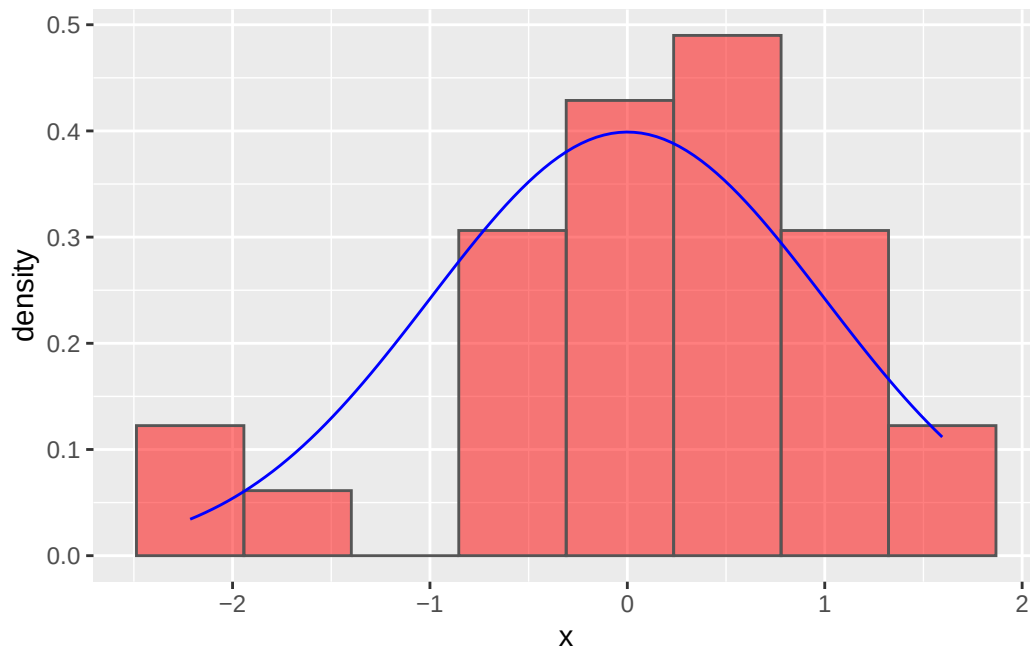


Il parametro `binwidth` fissa la larghezza degli intervalli; in alternativa si può fissare il loro numero con `bins`. La scelta di questi parametri influenza molto l'aspetto dell'istogramma, soprattutto per campioni piccoli: è bene provare più valori.

Spesso è utile rappresentare in ordinata la **densità** invece del conteggio, in modo che l'area totale delle barre sia pari a 1 e l'istogramma sia confrontabile con una densità di proba-

bilità teorica. Si ottiene con `y=after_stat(density)` (cioè: usa per *y* la densità *calcolata* da `geom_histogram()`). Possiamo così sovrapporre la PDF della normale standard con `geom_function()` (che vedremo meglio nella prossima sezione):

```
tibble(x = c4) %>%  
  ggplot(aes(x=x)) +  
  geom_histogram(aes(y=after_stat(density)),  
    bins=8,  
    color=gray(1/3),  
    fill="red", alpha=0.5) +  
  geom_function(fun=dnorm, color="blue")
```



7 Funzioni di distribuzione e loro grafici

In R, per ogni distribuzione di probabilità sono disponibili quattro funzioni, il cui nome è composto da un **prefisso** che indica cosa calcolare e da una **radice** che identifica la distribuzione:

Prefisso	Significato	Esempio (normale)
d	densità di probabilità (PDF), $f(x)$	dnorm(x)
p	funzione di ripartizione (CDF), $F(x) = P(X \leq x)$	pnorm(x)
q	quantile, cioè l'inversa della CDF: $F^{-1}(p)$	qnorm(p)
r	generazione di numeri casuali	rnorm(n)

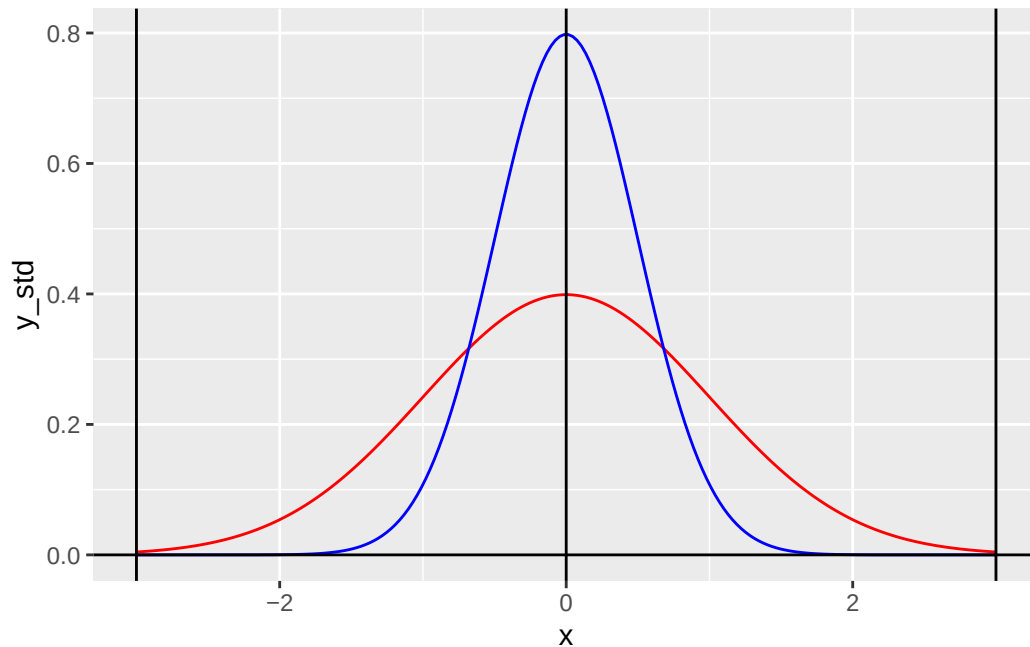
Le radici più comuni sono **norm** (normale), **t** (Student), **chisq** (χ^2), **f** (Fisher), **unif** (uniforme), **binom** (binomiale) e **pois** (Poisson). Ad esempio, **qchisq()** è la funzione quantile della χ^2 , e **dbinom()** è la funzione di probabilità della binomiale. Gli argomenti aggiuntivi sono i parametri della distribuzione (es. **mean** e **sd** per la normale, i gradi di libertà **df** per la *t*).

7.1 Densità di probabilità (PDF)

Per la distribuzione normale, la PDF è **dnorm(x, mean, sd)**; se **mean** e **sd** sono omessi, valgono rispettivamente 0 e 1 (normale standard). Costruiamo una tabella con un'ascissa fitta (200 punti) e alcune PDF calcolate su di essa:

```
data <- tibble(
  x = seq(-3, 3, length.out=200),
  y_std = dnorm(x),
  y = dnorm(x, sd=0.5),
  t = dt(x, 10)
)

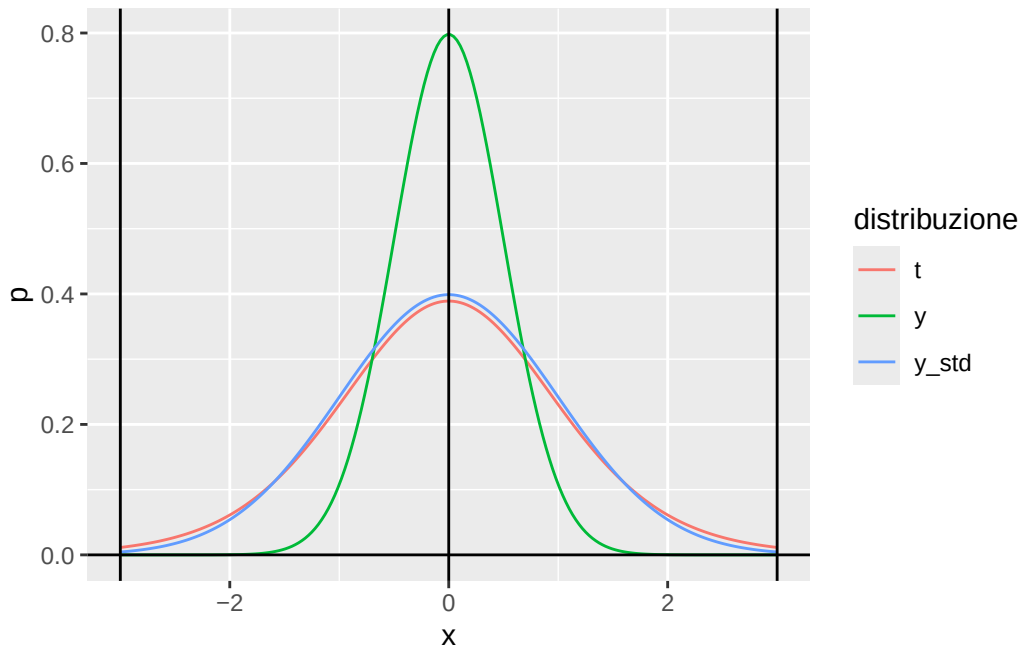
data %>%
  ggplot(aes(x=x)) +
  geom_line(aes(y=y_std), color="red") +
  geom_line(aes(y=y), color="blue") +
  geom_vline(xintercept = c(-3, 0, 3)) +
  geom_hline(yintercept = 0)
```



Le geometrie `geom_vline()` e `geom_hline()` tracciano linee verticali e orizzontali di riferimento, nelle posizioni indicate da `xintercept` e `yintercept`.

Questo approccio però ha un limite: serve un `geom_line()` per ogni curva, i colori vanno scelti a mano e non c'è una legenda. Invece che creare manualmente una serie per ogni curva, è più efficiente usare un pivot: portiamo la tabella in formato lungo (con `-x` impiliamo tutte le colonne *tranne* `x`) e associamo la colonna con il nome della distribuzione all'estetica `color`:

```
data %>%
  pivot_longer(-x, names_to="distribuzione", values_to="p") %>%
  ggplot(aes(x=x)) +
  geom_line(aes(y=p, group=distribuzione, color=distribuzione)) +
  geom_vline(xintercept = c(-3, 0, 3)) +
  geom_hline(yintercept = 0)
```



L'estetica `group` dice a `geom_line()` quali punti vanno uniti da una stessa linea (in questo caso sarebbe implicita in `color`, ma è bene conoscerla). Così facendo, aggiungere una nuova curva richiede solo di aggiungere una colonna alla tabella. Si noti come la t di Student con 10 gradi di libertà sia molto simile alla normale standard, ma con code leggermente più “pesanti”.

7.2 Ripartizione (CDF)

La funzione di ripartizione $F(x) = P(X \leq x)$ restituisce la probabilità che la variabile casuale assuma un valore minore o uguale a x , cioè l'area sotto la PDF a **sinistra** di x . Con l'opzione `lower.tail = FALSE` si ottiene invece l'area a **destra**, $P(X > x) = 1 - F(x)$:

```
pnorm(1.96) # area a sinistra di 1.96
```

```
[1] 0.9750021
```

```
pnorm(1.96, lower.tail = FALSE) # area a destra di 1.96
```

```
[1] 0.0249979
```

```
1 - pnorm(1.96) # equivalente
```

```
[1] 0.0249979
```

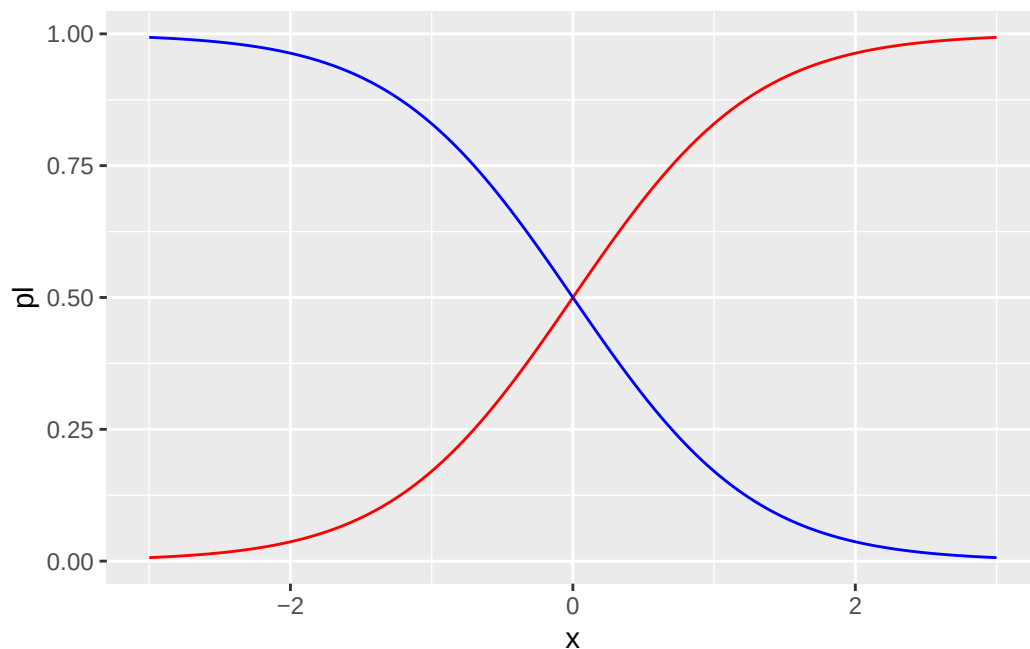
Questo è esattamente il calcolo che serve per ottenere un p -value. Ad esempio, per un test t bilaterale con statistica $t_0 = 2.3$ e 10 gradi di libertà:

```
t0 <- 2.3  
2 * pt(abs(t0), df=10, lower.tail = FALSE)
```

```
[1] 0.04425431
```

Graficamente, le due versioni della CDF sono simmetriche:

```
tibble(  
  x = seq(-3, 3, length.out=200),  
  pl = pt(x, 10, lower.tail = TRUE),  
  pu = pt(x, 10, lower.tail = FALSE)  
) %>%  
  ggplot(aes(x=x)) +  
  geom_line(aes(y=pl), color="red") +  
  geom_line(aes(y=pu), color="blue")
```



Si noti che qui la tabella viene creata con `tibble()` e passata direttamente a `ggplot()` con la pipe, senza salvarla in una variabile: è una pratica comune quando i dati servono solo per il grafico.

7.3 Quantile

La funzione quantile è l'**inversa** della CDF: data una probabilità p , restituisce il valore x_p tale che $P(X \leq x_p) = p$. È la funzione che si usa per calcolare i **valori critici** di un test o gli estremi di un intervallo di confidenza:

```
qnorm(0.975) # quantile della normale standard al 97.5%
```

```
[1] 1.959964
```

```
qnorm(0.025, lower.tail=FALSE) # stesso valore: area 0.025 a destra
```

```
[1] 1.959964
```

```
qt(0.975, df=10) # valore critico t per alpha=0.05, bilaterale
```

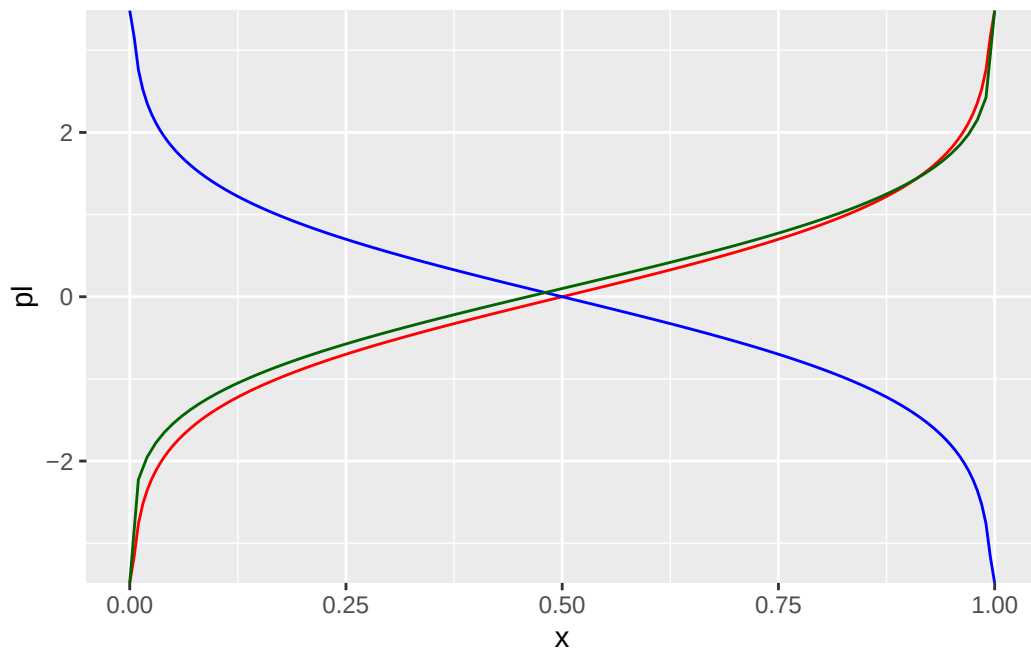
```
[1] 2.228139
```

```
pnorm(qnorm(0.975)) # q e p sono una l'inversa dell'altra
```

```
[1] 0.975
```

Il grafico della funzione quantile ha quindi in ascissa una probabilità (tra 0 e 1) e in ordinata un valore della variabile:

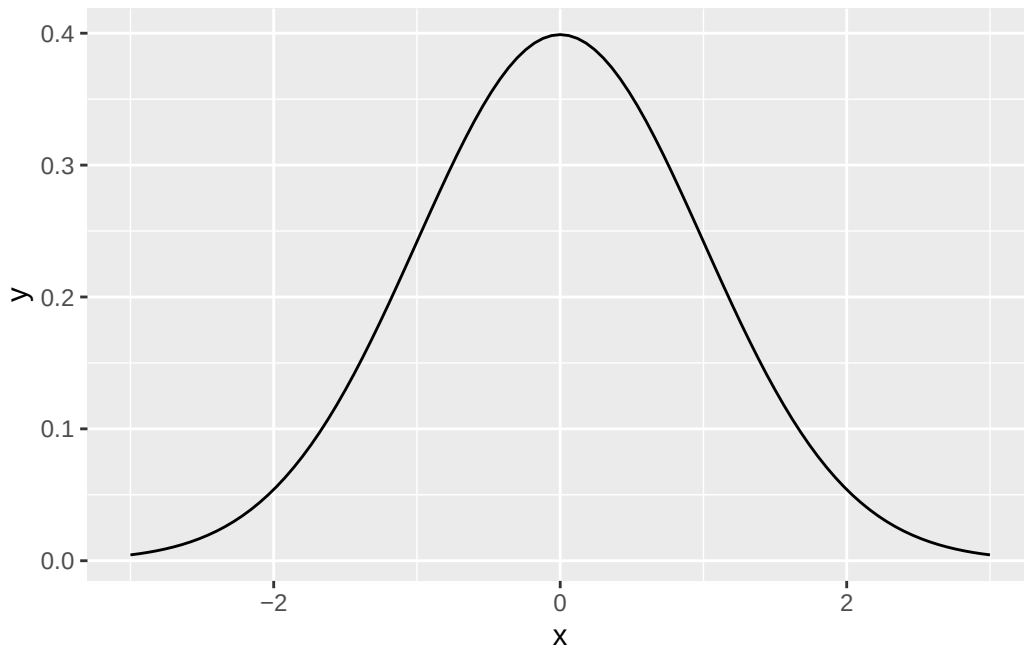
```
tibble(  
  x = seq(0, 1, length.out=200),  
  pl = qt(x, 10, lower.tail = TRUE),  
  pu = qt(x, 10, lower.tail = FALSE)  
) %>%  
  ggplot(aes(x=x)) +  
  geom_line(aes(y=pl), color="red") +  
  geom_line(aes(y=pu), color="blue") +  
  geom_function(fun = qnorm, color="darkgreen", args=list(mean=0.1))
```



Si osservi che per $p \rightarrow 0$ e $p \rightarrow 1$ il quantile tende a $\mp\infty$, e che la curva verde (quantile di una normale con media 0.1) è stata aggiunta con `geom_function()`, senza calcolarla nella tabella.

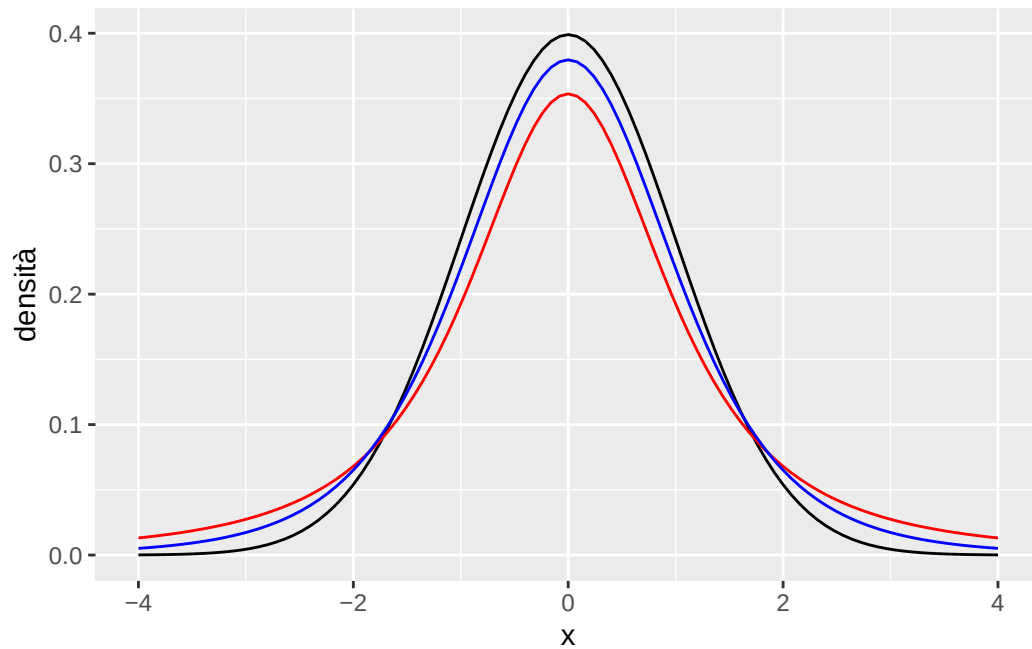
Se si vuole solo plottare una funzione, si può anche fare a meno di costruirsi una tabella dati e usare l'apposita geometria `geom_function()`, che vuole il *nome* della funzione da plottare (senza argomenti né parentesi!) e opzionalmente l'intervallo dell'ascissa:

```
ggplot() + geom_function(fun=dnorm, xlim=c(-3, 3))
```



Gli eventuali parametri della funzione (diversi dalla x) vanno passati come lista nell'argomento **args**. Ad esempio, per confrontare la t di Student con diversi gradi di libertà con la normale standard:

```
ggplot() +  
  geom_function(fun=dnorm, xlim=c(-4, 4), color="black") +  
  geom_function(fun=dt, xlim=c(-4, 4), args=list(df=2), color="red") +  
  geom_function(fun=dt, xlim=c(-4, 4), args=list(df=5), color="blue") +  
  labs(x="x", y="densità")
```



All'aumentare dei gradi di libertà la t di Student tende alla normale standard.

💡 Esercizio

dopo aver studiato l'esempio sopra riportato per il test di Student mediante la funzione `t.test()`, e dopo aver riletto le slide di teoria relative, implementare una funzione tale che:

```
t_test <- function(x, y, var.equal=TRUE, alpha=0.05) {  
  # restituisco una lista con le componenti:  
  list(  
    t0 = ...,  
    p.value = ...,  
    nu = ...,  
    mean = c(m1, m2),  
    conf.int = c(..., ...) )  
}
```

8 Accesso a file

8.1 Caricamento dati

8.2 Salvataggio dati

9 Gestione tabelle

9.1 Filtraggio

9.2 Riordino

9.3 Selezione

9.4 Modifica

9.5 Raggruppamento

9.6 Trasposizione